



Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance measure, Environment, Actuators, Sensors.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width` and `self.height` define the grid dimensions. `self.agent_pos`, `self.walls`, `self.food_positions`, and `self.opponents` represent the positions of the agent, obstacles, food, and opponents. `self.score` tracks performance; `self.steps` and `self.collision` track episode progress and collision status.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The baseline class defaults to two adversarial opponents, so that configuration is multi-agent. Each opponent moves independently and can collide with the controlled agent. However, the supplied runnable demo explicitly sets `num_opponents=0`, making that particular run single-agent.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of everything an agent has perceived through its sensors up to the present time. One call to `get_percept()` provides the current percept, not the complete history.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

Sensors: it supplies observations of the environment to the agent.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

The baseline is partially observable. The dictionary exposes the agent position, opponent positions, local food detection, wall and collision indicators, score, and the number of remaining food items. It does not expose all food locations or the wall layout. Seeing the entire canvas as a human does not give the agent that information. Also, `hit_wall` checks the current position even though wall entry is blocked, so it does not reliably report an attempted wall collision.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The Performance measure is updated through `self.score` when an attempted move hits a wall.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Performance should measure success at the task, such as collecting food and avoiding hazards. Rewarding internal processing alone could reward computation that does not improve outcomes. Resource use can be included when it is an explicit task objective, but it should not replace measuring task success.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

Hidden traps make safety-relevant state unavailable to the agent. An otherwise fully observable environment would become partially observable. This starter is already partially observable, so hiding traps increases the hidden information without changing that classification.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

The sensor identifies danger at the current location. A policy can use this information to leave the trap and, if it remembers observed positions, avoid returning to known traps. This can reduce expected score penalties. The sensor does not reveal nearby traps or prevent the first entry penalty, and adding a sensor alone does not change the starter's random action policy.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The standard example is a vacuum rewarded for the amount of dirt it cleans: it could dump dirt back onto the floor and clean it repeatedly to gain reward. A better measure rewards keeping the environment clean over time. This is the standard textbook example; the exact Lecture 01 wording has not been verified against a supplied lecture file in this task.

